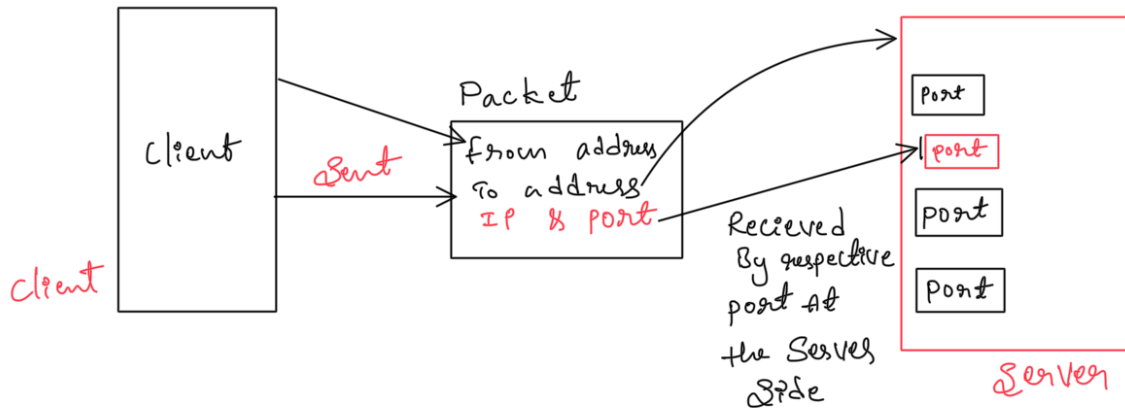


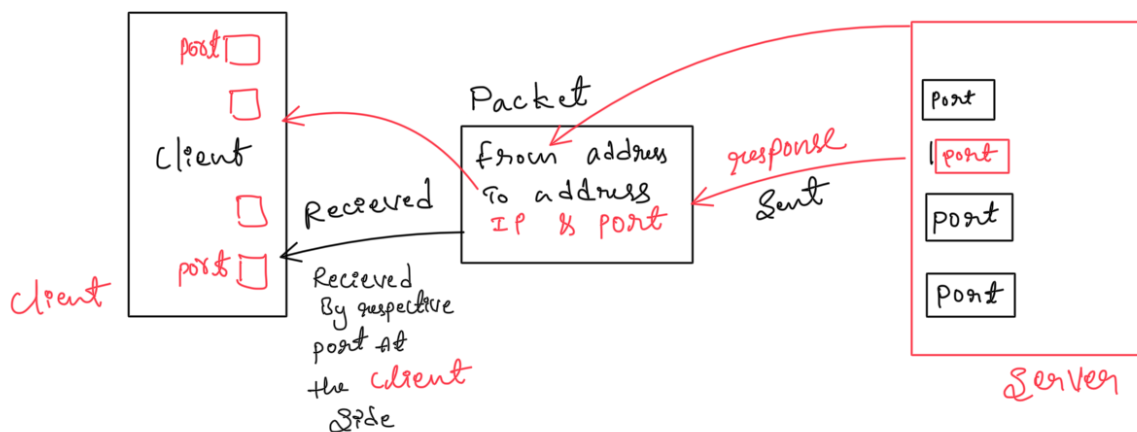
How Server Responses for a Request



Step 1 : Client Sends a packet to the Server with IP & port Number.

Step 2 : Server Responds Back after all checks and packet on request is received and response will be sent back in the packet to the Client address

NOTE : From address is Server address and To address is client address while responding back



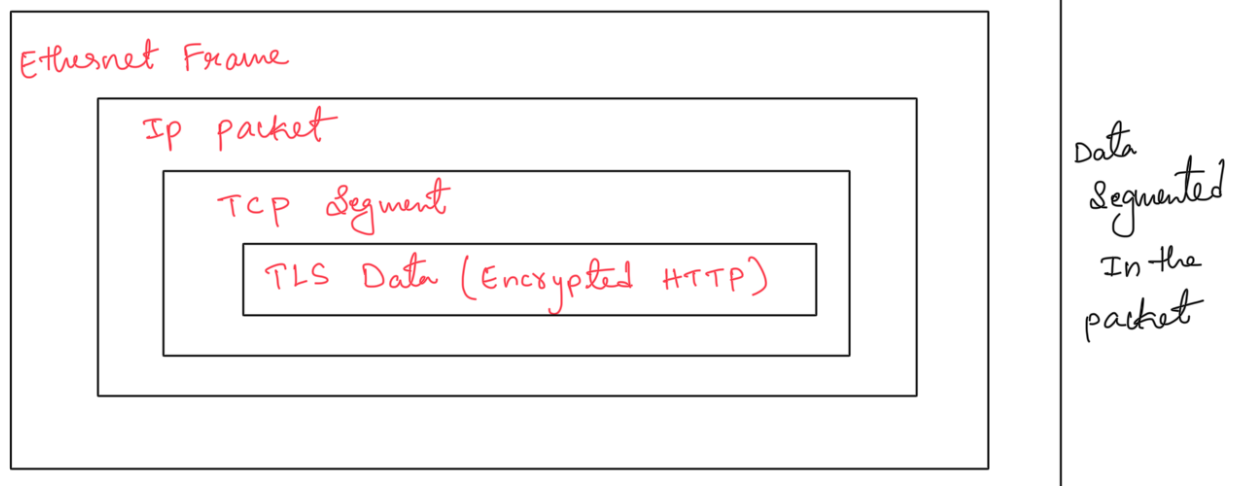
Packet encapsulation (nested envelopes)

Each lower layer wraps the higher layer's data

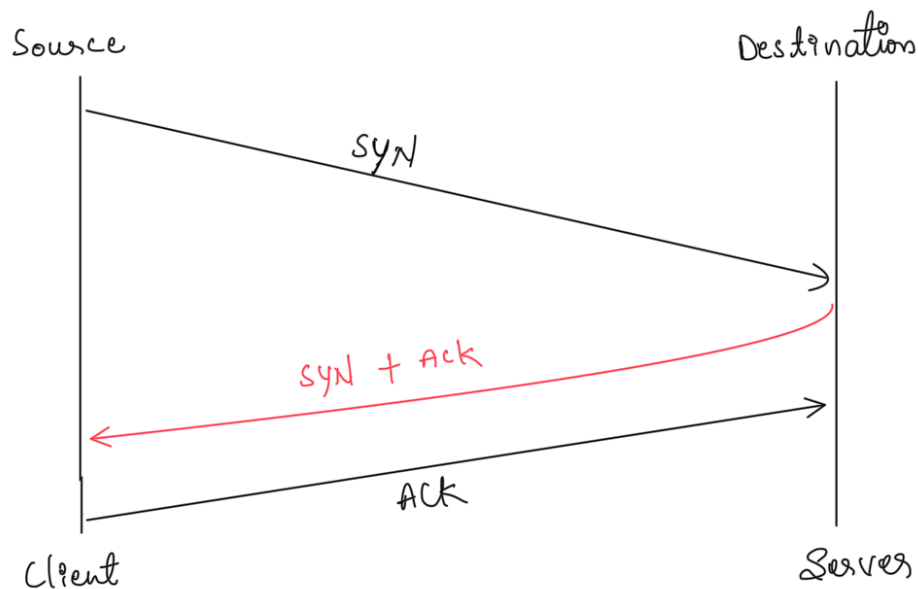
1. HTTP → plaintext request (eg GET /...)

2. TLS $\rightarrow$ Encrypts HTTP $\rightarrow$ TLS Application Data
3. TCP $\rightarrow$ adds src/dst ports, seq/ack, flags
4. IP $\rightarrow$ adds src/dst IP addresses, TTL
5. Ethernet / Wi-Fi $\rightarrow$ adds MAC addresses (and trailers)
6. Physical $\rightarrow$ turns frames into signals

ASCII (outer - inner)



HOW connection is established (3 way Handshake)



NOTES: Networking Fundamentals – HTTP, UDP, Ports, Firewalls, Routing, Wireshark & Code Demos

Beginner-friendly, structured notes for the **NOTES Project**. Includes definitions, analogies, diagrams (textual), and **working Java & Python code** (plus small C where appropriate).

Designed for step-by-step classroom use.

Index

1. [How to use these notes](#)
 2. [Learning objectives](#)
 3. [Key terms \(quick glossary\)](#)
 4. [OSI model overview](#)
 5. [Packet encapsulation \(nested envelopes\)](#)
 6. [Packet decapsulation \(unwrapping\)](#)
 7. [Why networks? A tiny history & motivation](#)
 8. [IP addressing & routing \(NAT, TTL, BGP\)](#)
 9. [Transport protocols: TCP vs UDP \(and QUIC\)](#)
 10. [Ports & sockets \(how OS/NIC keep track\)](#)
 11. [Firewalls, proxies & port-blocking](#)
 12. [How a request packet is formed \(headers\)](#)
 13. [End-to-end journey: Bengaluru → Singapore → back](#)
 14. [Sample packet anatomy \(request & response\)](#)
 15. [Wireshark: install & observe](#)
 16. [Code demos \(HTTP & UDP\): Java • Python • C](#)
 17. [Lab exercises \(hands-on\)](#)
 18. [FAQs & quick answers](#)
 19. [Pitfalls & best practices](#)
 20. [Must watch YouTube Videos](#)
 21. [Summary \(Quick Reference Table\)](#)
-

How to use these notes

- Read sections **4 → 6** to cement the mental model (OSI + encapsulation/decapsulation).
- Study **9 → 12** for protocol details.
- Walk through **13 → 15** to connect theory to real packets.
- Run **16 → 17** on your laptop for hands-on practice.
- Revisit **18 → 20** to review and self-check.

[Back to top](#)

Learning objectives

By the end, you should be able to:

- Explain **OSI layers** and **encapsulation/decapsulation**.
- Compare **TCP vs UDP** (and where **QUIC** fits).
- Describe **ports, sockets, NAT**, and how multiple clients share the same server port.
- Sketch the **life of a packet** (browser → OS → NIC → ISP → submarine cable → DC → app → back).
- Use **Wireshark** to observe handshakes and application data.
- Write minimal **HTTP** and **UDP** clients in **Java** and **Python** (plus a small **C** demo).

[Back to top](#)

Key terms (quick glossary)

- **Protocol**: Agreed rules for communication.
- **TCP**: Reliable, ordered, connection-oriented transport.
- **UDP**: Connectionless, best-effort, low-latency transport.
- **QUIC**: Modern transport over UDP with built-in security & streams.
- **Port**: Number identifying the receiving application on a host.
- **Socket**: Endpoint defined by (Protocol, IPs, Ports).
- **NAT**: Network Address Translation, maps private ↔ public flows.
- **Anycast**: Same IP prefix advertised from multiple locations; you reach the nearest.
- **BGP**: Inter-domain routing protocol of the Internet.
- **TLS**: Crypto layer for confidentiality/integrity (HTTPS).

[Back to top](#)

OSI model overview

OSI Layer	What it does	Examples
7 Application	App semantics	HTTP, HTTPS, DNS, SMTP
6 Presentation	Format, encryption	TLS/SSL, JSON, JPEG
5 Session	Conversations	TLS session state, RPC
4 Transport	End-to-end delivery	TCP, UDP , QUIC
3 Network	Global addressing/routing	IP (v4/v6), ICMP
2 Data Link	Local delivery	Ethernet , Wi-Fi, ARP
1 Physical	Signals/media	Copper, Fiber, Radio

Mapping: HTTP→L7, TLS→L6, TCP/UDP/QUIC→L4, IP→L3, Ethernet/Wi-Fi→L2, Fiber/Radio→L1.

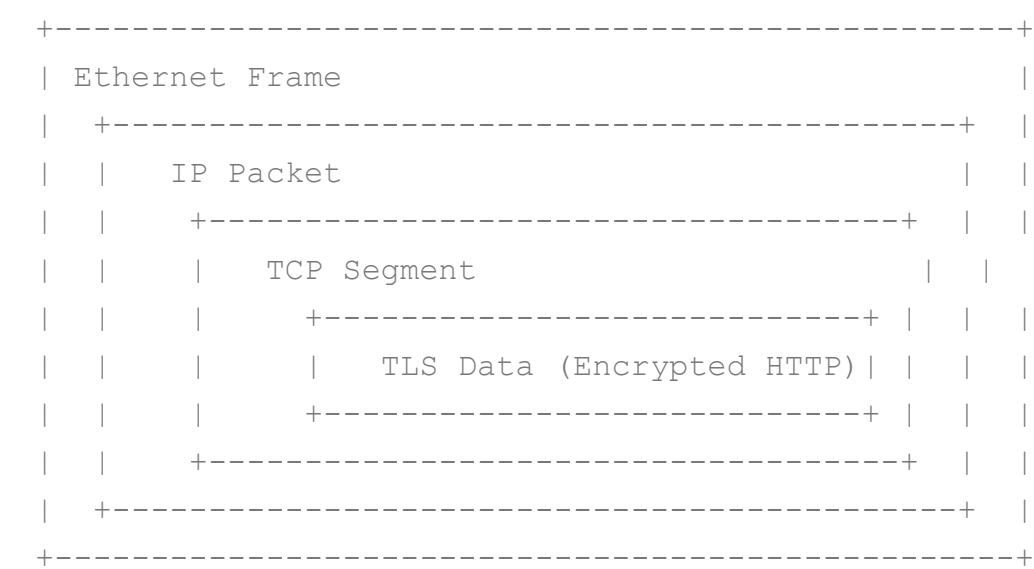
[Back to top](#)

Packet encapsulation (nested envelopes)

Each lower layer **wraps** the higher layer’s data:

1. **HTTP** → plaintext request (e.g., `GET /...`).
2. **TLS** → encrypts HTTP → *TLS Application Data*.
3. **TCP** → adds src/dst **ports**, seq/ack, flags.
4. **IP** → adds src/dst **IP addresses**, TTL.
5. **Ethernet/Wi-Fi** → adds **MAC** addresses (and trailer).
6. **Physical** → turns frames into signals.

ASCII (outer→inner):



Packet decapsulation (unwrapping)

At the receiver, layers **remove** their headers in reverse:

Signal → **Ethernet** → **IP** → **TCP** → **TLS** → **HTTP** → Browser.

- NIC checks **MAC**, kernel checks **IP & port**, TCP reassembles bytes, TLS decrypts, browser renders.

Incoming Signal → Ethernet → IP → TCP → TLS → HTTP → Browser

Why networks? A tiny history & motivation

- Early machines were isolated; sharing needed **rules (protocols)**.
- Two transport families emerged: **TCP** (reliable) & **UDP** (fast/minimal).
- A famous early milestone: **Oct 29, 1969**, ARPANET “LO” transmission (attempted “LOGIN”) from UCLA → SRI.

IP addressing & routing (NAT, TTL, BGP)

- **IP** gives global addresses; routers forward by **longest-prefix match**.
- **TTL** decrements at each hop to kill loops.
- **BGP** stitches providers’ networks so packets find viable paths.
- **NAT** maps (privateIP:port) ↔ (publicIP:port) and remembers each flow so replies return to the right LAN host.
- **Anycast** lets big services (e.g., Google) direct you to the **nearest** edge.

Transport protocols: TCP vs UDP (and QUIC)

TCP (reliable, connection-oriented)

- **3-way handshake:** SYN → SYN/ACK → ACK.
- Ordered, reliable stream; congestion control; teardown with FIN.
- HTTPS = **TCP + TLS + HTTP**.

UDP (connectionless)

- **No handshake;** a single datagram can be sent immediately.
- No ordering/retransmission; great for DNS, streaming, games.
- Apps can add their own reliability if needed.

QUIC (over UDP)

- Encrypts/handshakes like TLS but at transport level; multiplexed streams; basis for **HTTP/3**.

[Back to top](#)

Ports & sockets (how OS/NIC keep track)

Analogy: Host = **apartment building**, **port** = apartment number.

- OS keeps **socket tables** in RAM:
 - **Listening:** (proto, localIP, localPort)
 - **Established:** (proto, srcIP, srcPort, dstIP, dstPort) ← the **5-tuple**
- Multiple clients can connect to the **same server port** (e.g., 443) because each flow's 5-tuple is unique.
- **Two apps same port?** Generally **no**, unless: different IPs/protocols or special `SO_REUSEPORT` on supported OSes.
- NIC receives frames; **demultiplexing by port** is done by the **kernel**, often across CPU cores (RSS).

[Back to top](#)

Firewalls, proxies & port-blocking

- **Firewall:** rule engine (stateful/stateless) that allows/blocks by IP/port/proto/state.
- **Forward proxy:** client-side intermediary (policy, caching).
- **Reverse proxy/LB:** server-side front door (TLS termination, routing, scaling).
- **Security tip:** Close/deny unused ports; allow only what you need.

How a request packet is formed (headers)

- **IP (v4)**: src IP, dst IP, TTL, protocol (6=TCP, 17=UDP), checksum.
- **TCP**: src/dst ports, seq/ack numbers, flags (SYN/ACK/FIN/RST), window, options.
- **UDP**: src/dst ports, length, checksum.
- **HTTP**: start line + headers + body (plaintext over HTTP, encrypted inside TLS for HTTPS).

End-to-end journey: Bengaluru → Singapore → back

Story (HTTPS over TCP):

1. Browser resolves DNS (often UDP/53 or DoH/HTTPS).
2. OS opens socket to server IP:443; **TCP handshake**, then **TLS handshake**.
3. OS builds **TLS→TCP→IP→Ethernet**; driver hands to NIC; Wi-Fi/Ethernet signals go to **home router**.
4. **NAT** rewrites private→public; ISP → **backbone fiber** → **submarine cable** to **Singapore POP**.
5. Edge firewall/LB receive; kernel routes to app server; app computes response.
6. Return trip follows Internet routing back; NAT maps to your LAN host; kernel → browser renders.

ASCII path:

```
Browser → OS sockets → Kernel TCP/IP → NIC/Driver → Home Router (NAT)
→ ISP Edge → Backbone/Optical → Submarine Cable → Singapore POP
→ Firewall → Load Balancer → App Server
→ (Response back the chain) → Browser renders
```

Sample packet anatomy (request & response)

Request (client → server)

```
Ethernet:  Src=YourNIC, Dst=GatewayMAC, EtherType=IPv4
IP:        Src=203.0.113.5, Dst=142.250.x.y, TTL=58, Proto=TCP
```


TCP: SrcPort=53124, DstPort=443, Flags=PSH,ACK, Seq, Ack, Options
TLS: Application Data (encrypted HTTP GET /search?q=...)

Response (server → client)

Ethernet: Src=UpstreamMAC, Dst=YourNIC, EtherType=IPv4
IP: Src=142.250.x.y, Dst=203.0.113.5, TTL=53, Proto=TCP
TCP: SrcPort=443, DstPort=53124, Flags=PSH,ACK, Seq, Ack
TLS: Application Data (encrypted HTTP 200 OK + HTML/JSON)

[Back to top](#)

Wireshark: install & observe

Install:

- Windows: Wireshark + **Npcap** (accept prompt).
- macOS: `brew install --cask wireshark` (allow capture).
- Ubuntu/Debian: `sudo apt install wireshark` then `sudo usermod -aG wireshark $USER` (re-login).

Filters:

- `tcp.port == 443` (HTTPS), `udp.port == 53` (DNS), `icmp` (ping).

What you'll see (HTTPS):

1. TCP: **SYN** → **SYN/ACK** → **ACK**
2. TLS 1.3: **ClientHello** → **ServerHello (+cert)** → **Finished**
3. TLS: **Application Data** (encrypted request/response)

Open a terminal: `curl -v https://www.google.com` and watch in Wireshark.

[Back to top](#)

Code demos (HTTP & UDP): Java • Python • C

Minimal, readable, with timeouts. Run on a machine with Internet access.

Java – HTTP GET (Java 11+ `HttpClient`)

```

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

/**
 * Simple HTTPS GET using Java 11+ HttpClient.
 * Prints status and first 200 chars of body.
 */
public class SimpleHttpGet {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newBuilder()
            .build();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://httpbin.org/get"))
            .GET()
            .build();

        HttpResponse<String> response =
            client.send(request, HttpResponse.BodyHandlers.ofString())

        System.out.println("Status: " + response.statusCode());
        String body = response.body();
        System.out.println("Body (truncated): " + body.substring(0, Math.
    }
}

```

Python – HTTP GET (standard library)

```

"""
Simple HTTPS GET using urllib (no external deps).
Prints status code and first 200 chars of response.
"""
import urllib.request

def http_get():
    url = "https://httpbin.org/get"
    with urllib.request.urlopen(url, timeout=10) as resp:
        status = resp.getcode()
        body = resp.read().decode("utf-8", errors="replace")
        print("Status:", status)

```

```

        print("Body (truncated):", body[:200])

if __name__ == "__main__":
    http_get()

```

Java – UDP DNS query (minimal, UDP/53)

```

import java.net.*;
import java.nio.ByteBuffer;
import java.util.Arrays;

/**
 * Minimal DNS A-record query for example.com over UDP to 8.8.8.8.
 * Prints first 40 response bytes in hex.
 */
public class SimpleDnsUdp {
    private static byte[] buildDnsQuery(String host) throws Exception {
        ByteBuffer header = ByteBuffer.allocate(12);
        header.putShort((short) 0x1234).putShort((short) 0x0100) // RD=1
            .putShort((short) 1).putShort((short) 0)
            .putShort((short) 0).putShort((short) 0);

        ByteBuffer qname = ByteBuffer.allocate(256);
        for (String label : host.split("\\.")) {
            byte[] b = label.getBytes("UTF-8");
            qname.put((byte) b.length).put(b);
        }
        qname.put((byte) 0x00);

        ByteBuffer tail = ByteBuffer.allocate(4);
        tail.putShort((short) 1).putShort((short) 1); // QTYPE=A, QCLASS=

        byte[] query = new byte[12 + qname.position() + 4];
        System.arraycopy(header.array(), 0, query, 0, 12);
        System.arraycopy(qname.array(), 0, query, 12, qname.position());
        System.arraycopy(tail.array(), 0, query, 12 + qname.position(), 4);
        return query;
    }

    public static void main(String[] args) throws Exception {
        byte[] query = buildDnsQuery("example.com");
        try (DatagramSocket socket = new DatagramSocket()) {

```

```

        socket.setSoTimeout(3000);
        DatagramPacket req = new DatagramPacket(query, query.length,
            InetAddress.getByName("8.8.8.8"), 53);
        socket.send(req);

        byte[] buf = new byte[512];
        DatagramPacket resp = new DatagramPacket(buf, buf.length);
        socket.receive(resp);

        byte[] data = Arrays.copyOf(resp.getData(), resp.getLength())
        for (int i = 0; i < Math.min(data.length, 40); i++) {
            System.out.printf("%02x ", data[i]);
        }
    }
}

```

Python – UDP DNS query (minimal)

```

"""
Minimal DNS A-record query for example.com over UDP to 8.8.8.8:53.
Prints first 40 bytes of response in hex.
"""
import socket

def build_dns_query(hostname: str) -> bytes:
    tid = b"\x12\x34"          # Transaction ID
    flags = b"\x01\x00"        # Standard query, RD=1
    counts = b"\x00\x01\x00\x00\x00\x00\x00\x00" # QD=1, others=0
    header = tid + flags + counts
    qname = b"".join(bytes([len(l)]) + l.encode() for l in hostname.split)
    tail = b"\x00\x01\x00\x01" # QTYPE=A, QCLASS=IN
    return header + qname + tail

def dns_udp_query():
    server = ("8.8.8.8", 53)
    query = build_dns_query("example.com")
    with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
        s.settimeout(3.0)
        s.sendto(query, server)
        data, _ = s.recvfrom(512)
        print("Response (hex, first 40 bytes):", data[:40].hex(" "))

```

```
if __name__ == "__main__":
    dns_udp_query()
```

C – UDP DNS (example.com)

```
/* Minimal UDP DNS query to 8.8.8.8:53 for A record of example.com */
#include <arpa/inet.h>
#include <netinet/in.h>
#include <stdio.h>
#include <string.h>
#include <sys/socket.h>
#include <unistd.h>

int main() {
    unsigned char buf[512];
    unsigned char query[64];
    int qlen = 0;

    // Header: ID=0x1234, flags=0x0100 (RD), QDCOUNT=1
    unsigned char hdr[] = {0x12,0x34, 0x01,0x00, 0x00,0x01, 0x00,0x00, 0x00,0x00};
    memcpy(query + qlen, hdr, sizeof(hdr)); qlen += sizeof(hdr);
    // QNAME: "example.com"
    const char* name = "example.com";
    const char* p = name;
    while (*p) {
        const char* dot = strchr(p, '.');
        int len = dot ? (dot - p) : (int)strlen(p);
        query[qlen++] = (unsigned char)len;
        memcpy(query + qlen, p, len); qlen += len;
        p += len + (dot ? 1 : 0);
        if (!dot) break;
    }
    query[qlen++] = 0x00;
    // QTYPE=A(1), QCLASS=IN(1)
    unsigned char tail[] = {0x00,0x01, 0x00,0x01};
    memcpy(query + qlen, tail, sizeof(tail)); qlen += sizeof(tail);

    int s = socket(AF_INET, SOCK_DGRAM, 0);
    struct sockaddr_in dst = {0};
    dst.sin_family = AF_INET;
    dst.sin_port = htons(53);
```

```

inet_pton(AF_INET, "8.8.8.8", &dst.sin_addr);

sendto(s, query, qlen, 0, (struct sockaddr*)&dst, sizeof(dst));
struct sockaddr_in from; socklen_t flen = sizeof(from);
int n = recvfrom(s, buf, sizeof(buf), 0, (struct sockaddr*)&from, &flen);
close(s);

if (n > 0) {
    for (int i = 0; i < n && i < 40; i++) printf("%02x ", buf[i]);
    printf("\n");
}
return 0;
}

```

[Back to top](#)

Lab exercises (hands-on)

1. **DNS over UDP:** `nslookup example.com` or `dig example.com`, filter `udp.port == 53` in Wireshark.
2. **HTTPS over TCP:** `curl -v https://www.google.com`, filter `tcp.port == 443`; spot **SYN/SYN-ACK/ACK** and **TLS ClientHello/ServerHello**.
3. **Trace route:** `tracert www.google.com` (Linux/macOS) / `tracert` (Windows); discuss TTL.
4. **Observe ports:** `netstat -an | grep ESTABLISHED` (Linux/macOS) / `find "ESTABLISHED"` (Windows).
5. **Firewall idea:** try `telnet google.com 21` (FTP); connection typically refused/blocked.
6. **Custom UDP:** run the Python/Java UDP DNS code; filter `udp.port == 53` to see your query/response.

[Back to top](#)

FAQs & quick answers

- **Can two apps use the same port?** Not on the same IP/protocol, unless `SO_REUSEPORT` or different IPs/protocols.
- **How does a server reply to the right client on one port?** Kernel tracks each connection by **5-tuple**; replies use that state.
- **Is there a UDP handshake? No.** Apps may add their own (e.g., QUIC crypto handshake).

- **Why block ports?** Reduce attack surface; allowlist only what you need.

[Back to top](#)

Pitfalls & best practices

- Prefer **HTTPS** (TLS).
- Set **timeouts** and retry idempotent requests.
- Validate inputs; handle partial reads/writes.
- Keep **unused ports closed**; use stateful firewalls.
- Observe with **Wireshark**; verify your mental model with real packets.

[Back to top](#)

Must watch YouTube Videos

Network Packets (Theory ↔ Real-World)

Understanding networks isn't just about definitions—it's about **seeing real packets move**. These two short videos perfectly **bridge theory and practice** so the OSI layers, ports, NAT, routing, TCP/TLS handshakes, and CDNs **click**.

Videos

1. Journey of a Network Packet

https://youtu.be/-g3I_ZNhePM

Follow a packet end-to-end and relate OSI layers to what actually happens on wires, Wi-Fi, routers, and data centers.

2. A Packet's Tale: How Does the Internet Work?

https://youtu.be/ewrBalT_eBM

Ride shotgun with a packet from your fingertips, through circuits/wires/cables, to a host server and back, all in <1s.

[Back to top](#)

Summary (Quick Reference Table)

Topic	One-liner	Jump
OSI model	7 layers from Physical → Application	Go
Encapsulation	Wraps data as it goes down the stack	Go
Decapsulation	Unwraps data as it goes up the stack	Go
TCP vs UDP vs QUIC	Reliable vs best-effort vs modern UDP-based	Go
Ports & sockets	OS uses 5-tuple to demultiplex flows	Go
Firewalls/Proxies	Enforce policy; reverse proxies front apps	Go
Packet headers	IP/TCP/UDP/HTTP essentials	Go
Journey BLR→SIN	Real path of an HTTPS request/response	Go
Packet anatomy	Minimal request/response breakdown	Go
Wireshark	Capture & filter to see real handshakes	Go
Code demos	Java/Python/C for HTTP/UDP	Go
Labs	Hands-on tasks to cement learning	Go
FAQs	Short answers to common doubts	Go
Best practices	Security/timeouts/observability	Go
Videos	Two must-watch intros	Go

[Back to top](#)